

Lý thuyết Chuyên sâu - Phần 1: Cấu trúc Đồ thị Tri thức (Knowledge Graph Structure)

Trong hệ thống WebPersonalLearning, “Cây Tri Thức” thực chất là một **Đồ thị có hướng không chu trình (DAG - Directed Acyclic Graph)**. Mô hình DAG cho phép biểu diễn các mối quan hệ đa tuyến tính (một bài học có thể yêu cầu nhiều bài học tiên quyết, và là nền tảng cho nhiều bài học tiếp theo).

1.1. Mô hình Toán học của Đồ thị (Mathematical Model)

Ký hiệu hệ thống tri thức là một đồ thị $G = (V, E)$, trong đó: - V (**Vertices**) là tập hợp tất cả các đơn vị tri thức (Nodes). - E (**Edges**) là tập hợp các mối quan hệ tiên quyết. Một cạnh có hướng $e = (u, v) \in E$ có nghĩa là node u là điều kiện tiên quyết bắt buộc để mở khóa node v .

Vì quá trình học tập là quá trình tích lũy tiến lên phía trước, đồ thị này tuyệt đối không được có chu trình (Acyclic). Nếu $u \rightarrow \dots \rightarrow v$, thì không thể có đường đi từ $v \rightarrow u$.

Phân rã Tập đỉnh (Vertex Partitioning)

Tập đỉnh V được phân rã thành 3 tập hợp con rời rạc (disjoint sets):

$$V = V_{macro} \cup V_{micro} \cup V_{assess}$$

1. V_{macro} (**Macro Nodes**):

- Đóng vai trò là các tập hợp nhãn (Label Sets) hoặc các Hyper-nodes chứa các subgraph.
- Về mặt toán học, mỗi $v \in V_{macro}$ ánh xạ tới một đồ thị con $G' \subset G$. Nó biểu diễn một “Chương” hay “Mảng chủ đề”.

2. V_{micro} (**Micro Nodes**):

- Là các lá (leaves) hoặc các điểm trung chuyển của đồ thị thực thi.
- Đây là nơi chứa nội dung học tập thực tế và dữ liệu tương tác của user. Tải trọng (payload) của đồ thị nằm ở tập hợp này.

3. V_{assess} (**Assess Nodes - Nút đánh giá**):

- Đóng vai trò là các **Gateway Nodes (Nút rào chắn)**.
 - Tính chất đồ thị: Giả sử đồ thị con của Chương 1 là G_1 và Chương 2 là G_2 . Một node $a \in V_{assess}$ được định nghĩa là một Cut-vertex (đỉnh cắt) hoặc cầu nối bắt buộc. Mọi đường đi (path) từ bất kỳ đỉnh nào trong G_1 đến bất kỳ đỉnh nào trong G_2 đều phải đi qua a .
 - Nghĩa là: Người học bị “kẹt” lại tại a cho đến khi vượt qua bài đánh giá tổng hợp.
-

1.2. Không gian Trạng thái của một Node (Node State Space)

Mỗi node $v \in V_{micro} \cup V_{assess}$ là một vector đặc trưng toán học chứa 3 tham số cốt lõi quyết định hành vi của hệ thống AI. Ký hiệu trạng thái của node v là S_v :

$$S_v = (\alpha_v, \rho_v, \mathcal{B}_v)$$

1. α_v : Độ khó nhận thức (Cognitive Alpha Base)

- **Định nghĩa:** Một đại lượng vô hướng định lượng độ trừu tượng và sự phức tạp của kiến thức.
- **Miền giá trị:** $\alpha_v \in \mathbb{Z} \cap [10,30]$.
- **Ý nghĩa hệ thống:** Alpha base thấp (10-15) dành cho các định nghĩa, sự kiện. Alpha base cao (20-30) dành cho các công thức phức tạp, tư duy thiết kế hệ thống. Giá trị này là hằng số được sinh ra bởi AI khi tạo cấu trúc ban đầu.

2. ρ_v : Vị trí tương đối trong Đồ thị (Position Ratio)

- **Định nghĩa:** Tỷ lệ cho biết người học đã đi được bao xa trong đồ thị khi chạm đến node v .
- **Toán học:** Để tính ρ_v , hệ thống thực hiện thuật toán **Sắp xếp Topo (Topological Sorting)** trên G để gán một chỉ số thứ tự $rank(v)$ cho từng node $v \in V_{micro}$.

$$\rho_v = \frac{rank(v) - 1}{|V_{micro}| - 1}$$

- **Miền giá trị:** $\rho_v \in [0,1]$.
- **Ý nghĩa hệ thống:** $\rho_v \approx 0$ nghĩa là nhập môn. $\rho_v \approx 1$ nghĩa là cuối khóa học. Hàm này được hiện thực hóa trong hàm `get_node_position_ratio()`.

3. $\mathcal{B}_v$: Hồ sơ Bloom mục tiêu (Target Bloom Profile)

- **Định nghĩa:** Một tập hợp con các bậc Bloom mà người học phải đạt được tại node đó. Tập hợp các bậc Bloom khả dĩ là $U = \{1,2,3,4,5,6\}$.
- **Toán học:** $\mathcal{B}_v \subset U$. Nó là kết quả của một hàm ánh xạ (mapping function) $f: (\alpha_v, \rho_v) \rightarrow \mathcal{B}_v$. (Chi tiết hàm f này sẽ được trình bày ở Phần 2).

1.3. Ánh xạ sang Hệ thống Thực tế (System Implementation)

Cấu trúc đồ thị DAG được số hóa thành cấu trúc JSON dạng Flat List kết hợp References để tối ưu hóa việc truy xuất dữ liệu trên ứng dụng Web:

```
{
  "course_name": "Tên khóa học",
  "macro_nodes": [
```

```

    { "id": "chap1", "title": "Chương 1" }
  ],
  "micro_nodes": [
    {
      "id": "c1.1",
      "chapter": "chap1",
      "alpha_base": 12,
      "bloom_profile": {
        "effective_levels": [1, 2]
      },
      "prerequisites": []
    },
    {
      "id": "c1.2",
      "chapter": "chap1",
      "alpha_base": 18,
      "prerequisites": ["c1.1"]
    }
  ],
  "assess_nodes": [
    {
      "id": "assess_chap1",
      "type": "assess",
      "prerequisites": ["c1.1", "c1.2"]
    }
  ]
}

```

Kiến trúc Dữ liệu

- Để giải quyết vấn đề duyệt cây (Tree Traversal), hệ thống chuyển đổi JSON này thành Flat Dictionary {"id": node_data} trong runtime qua hàm `_get_all_nodes_flat()`. Việc tra cứu một node và các thuộc tính của nó tốn độ phức tạp $O(1)$.
- Hệ thống Bloom State được lưu tách biệt khỏi cấu trúc cây (tại `user_data/{username}/bloom_state/{subject_id}.json`) để đảm bảo Graph là Immutable (không đổi), trong khi User State (trạng thái người học) là Mutable (thay đổi liên tục).

Lý thuyết Chuyên sâu - Phần 2: Thuật toán Bloom, Đánh giá và Mô hình Trí nhớ Ebbinghaus

Trong hệ thống Knowledge Tree, sự tương tác của học viên không chỉ là các thao tác nhấp chuột đơn thuần mà là một tập hợp các chuỗi dữ liệu (Data Streams). Hệ thống biến các luồng dữ liệu này thành các đại lượng toán học để chạy ba động cơ cốt lõi: **Động cơ Hiệu chuẩn (Calibration Engine)**, **Động cơ Đánh giá (Scoring Engine)**, và **Động cơ Trí nhớ (Memory Engine)**.

2.1. Động cơ Hiệu chuẩn Bloom (Bloom Calibration Engine)

Động cơ này giải quyết bài toán: *Với một đơn vị kiến thức cụ thể, não bộ học viên cần vận dụng đến tầng nhận thức nào?*

Hàm Hiệu chuẩn $B_{target}(\alpha, \rho)$: Gọi α là Độ khó sinh học (Cognitive Alpha Base) và ρ là Vị trí tương đối (Position Ratio) của node. Hệ thống áp dụng chuỗi hàm điều kiện (Piecewise Function) sau để tìm tập hợp bậc Bloom mục tiêu B_{base} :

$$B_{base}(\alpha) = \begin{cases} \{1,2\} & \text{khi } \alpha \leq 12 \\ \{1,2,3\} & \text{khi } 12 < \alpha \leq 18 \\ \{2,3,4\} & \text{khi } 18 < \alpha \leq 24 \\ \{3,4,5\} & \text{khi } \alpha > 24 \end{cases}$$

Heuristic thưởng tiến trình (Positional Heuristic): Khi học viên tiến gần đến cuối khóa học ($\rho \geq 0.85$), hệ thống cho rằng họ đã tích lũy đủ các khái niệm đơn lẻ và cần phải kết nối chúng. Một hàm thưởng được áp dụng:

$$B_{target} = \begin{cases} B_{base} \cup \{\min(6, \max(B_{base}) + 1)\} & \text{nếu } \rho \geq 0.85 \\ B_{base} & \text{nếu } \rho < 0.85 \end{cases}$$

(Thuật toán này được ánh xạ 1:1 trong hàm `calibrate_bloom_from_alpha` tại file `bloom_taxonomy.py`)

2.2. Động cơ Đánh giá và Mở khóa (Scoring & Progressive Unlock)

Sau khi B_{target} được xác định, hệ thống theo dõi hiệu suất trả lời (Accuracy) tại từng bậc i . Gọi s_i là độ chính xác tại bậc i ($s_i = \frac{correct_i}{total_i}$).

Phương trình tính điểm tổng Bloom (Overall Bloom Score)

Do các bậc cao của Bloom tốn nhiều năng lượng não bộ hơn, chúng mang trọng số W_i cao hơn. Trọng số được tính theo hàm tuyến tính: $W_i = 1.0 + (i - 1) \times 0.2$. Điểm tổng hợp được tính theo phương trình trung bình có trọng số:

$$Overall_Bloom = \frac{\sum_{i \in \text{Attempted}} (i \times s_i \times W_i)}{\sum_{i \in \text{Attempted}} W_i}$$

Phương trình này đảm bảo rằng: Dù học viên đạt 100% độ chính xác ở Bậc 1 (Ghi nhớ), điểm Overall của họ vẫn rất thấp so với việc đạt 60% độ chính xác ở Bậc 4 (Phân tích). Điểm tối đa giới hạn là 6.0.

Cổng Logic Mở khóa Tiến trình (Progressive Unlock Gate)

Người học không thể truy cập một bậc Bloom cao nếu nền tảng chưa vững. Bậc Bloom n được đánh giá là đã “Mở khóa” (True) thông qua biểu thức logic:

$$Unlocked(n) \Leftrightarrow (n = 1) \vee [(s_{n-1} \geq \tau_{n-1}) \wedge (|T_{n-1}| \geq 2)]$$

- τ_{n-1} : Là ngưỡng Pass Threshold của bậc $n - 1$ (Ví dụ: 0.70 cho Bậc 1, 0.60 cho Bậc 4).
- $|T_{n-1}|$: Số lượng loại hình bài tập khác nhau đã làm ở bậc $n - 1$ (Ví dụ MCQ, Flashcard). Ràng buộc $|T_{n-1}| \geq 2$ ép người học không được “học vẹt” chỉ thông qua việc đánh lại trắc nghiệm mà phải thực hành đa dạng.

2.3. Động cơ Trí nhớ Ebbinghaus (Memory Engine)

Để chống lại sự lãng quên tự nhiên của con người, hệ thống sử dụng biến thể số học của **Đường cong quên lãng Ebbinghaus (Ebbinghaus Forgetting Curve)**. Trạng thái trí nhớ tại một node được giám sát bởi Hệ số Duy trì $R(t)$ (Retention Factor).

Phương trình Phân rã (Decay Equation)

$$R(t, S) = e^{-\frac{t}{S}}$$

- t : Thời gian trôi qua kể từ lần học/ôn tập cuối (đơn vị: Giờ).
- S : Cường độ trí nhớ (Memory Strength). Trị số khởi tạo $S_0 = 2.0$. Giá trị S càng lớn, đồ thị R càng phẳng, người học càng lâu quên kiến thức.

Ứng dụng vào Hệ thống: Mỗi khi hệ thống load trạng thái Bloom của người học, điểm số thực tế sẽ bị trừ hao tự động bởi hệ số R :

$$\text{Decayed Score} = \text{Original Score} \times R(t, S)$$

Nếu R tụt xuống quá thấp, thanh tiến trình của người học sẽ giảm, kích hoạt trạng thái báo động đỏ (cần phải ôn tập).

Phương trình Cập nhật Cường độ (Strength Update Function)

Sau khi người học thực hiện Ôn tập (Review), độ chính xác của bài kiểm tra (P) sẽ được đưa vào hàm để cập nhật lại cường độ S . Điều này tương đương với thuật toán Spaced Repetition (SRS):

$$S_{new} = \begin{cases} \min(720, S_{old} \times 1.5) & \text{khi } P \geq 0.7 \\ \max(1.0, S_{old} \times 1.1) & \text{khi } 0.4 \leq P < 0.7 \\ \max(0.5, S_{old} \times 0.7) & \text{khi } P < 0.4 \end{cases}$$

Phân tích thuật toán: - Nếu bạn trả lời đúng ($P \geq 0.7$), trí nhớ của bạn được gia cố mạnh mẽ (tăng gấp 1.5 lần thời gian chống lãng quên). Max giới hạn ở 720 giờ (30 ngày).
- Nếu bạn trả lời sai ($P < 0.4$), hệ thống đánh giá rằng cấu trúc nơ-ron liên kết kiến thức này đang sụp đổ, S bị phạt giảm xuống ($0.7x$), vòng lặp ôn tập tiếp theo sẽ xuất hiện sớm hơn rất nhiều.

2.4. Động lực học (Gamification & XP Engine)

Hệ thống toán học Gamification dựa trên vòng lặp phản hồi (Feedback Loop) dương tính để tiết ra Dopamine cho người học:

$$XP_{earned} = XP_{base}(\text{Action}) \times M(\text{streak})$$

- XP_{base} : Biến thiên theo mức độ phức tạp của hành vi (Trả lời MCQ đúng = 10 XP; Giải quyết Socratic vấn đáp = 25 XP).
- $M(\text{streak})$: Hàm bậc thang (Step function) tăng dần theo chuỗi ngày liên tiếp. Trọng số dao động từ 1.0 (không có streak) đến 2.0 (streak 30 ngày). Điều này phạt rất nặng hành vi bỏ dở học tập, vì khi rút chuỗi, M quay về 1.0, lượng XP cày cuốc mất đi một nửa lợi thế.

Lý thuyết Chuyên sâu - Phần 3: Hệ thống Tương tác AI Tutor theo Thang Bloom

AI Tutor trong hệ thống Cây Tri Thức không phải là một mô hình ngôn ngữ tĩnh hay một Chatbot hỏi-đáp thông thường. Nó được thiết kế như một **Hệ thống Tác tử Nhận thức Ngữ cảnh (Context-Aware Agent System)**. Tại mỗi Node, AI sẽ đọc Vector Trạng thái của người học và tự động chuyển đổi “Vai diễn sư phạm” (Pedagogical Persona) để phù hợp với bậc Bloom mục tiêu.

3.1. Mô hình AI Nhận thức Ngữ cảnh (Context-Aware AI Model)

Mỗi khi người học mở một node, AI Tutor sẽ được khởi tạo với một System Prompt được lắp ghép động (Dynamically Assembled) từ 3 tham số: 1. **Nội dung Node (C_v)**: Dữ liệu kiến thức thô. 2. **Lịch sử Người học (H_u)**: Điểm số hiện tại, lịch sử sai lầm. 3. **Bậc Bloom Mục tiêu (B_{target})**: Tham số cốt lõi quyết định hành vi.

Việc tiêm B_{target} vào prompt ép AI từ bỏ thói quen “giải đáp hộ” (vốn là bản năng của LLM) để trở thành một người thầy thực thụ.

3.2. Ánh xạ Vai diễn Sư phạm (Pedagogical Persona Mapping)

Bậc 1 & 2: Ghi nhớ và Thấu hiểu (Remember & Understand)

- **Vai diễn của AI**: Người Kiểm Tra Trực Tiếp (The Examiner)
- **Chiến lược**: Ở tầng này, mục tiêu là chuyển đổi dữ liệu từ Trí nhớ làm việc (Working Memory) sang Trí nhớ ngắn hạn (Short-term Memory). Sự mơ hồ (Ambiguity) là kẻ thù.
- **Cách AI hoạt động**:
 - Sinh ra các dạng bài tập xác định (Deterministic) như MCQ, Flashcard, Matching.
 - Xây dựng các “Distractors” (Đáp án nhiễu) dựa trên những lỗi sai phổ biến nhất (Common Misconceptions) sinh ra từ lịch sử học tập.
- **Cơ chế Feedback**: Trực diện và ngay lập tức. Nếu sai, AI cung cấp câu trả lời đúng và giải thích ngắn gọn bằng 1-2 câu. Đánh giá tính đúng/sai dựa trên thuật toán Semantic Similarity với ngưỡng tin cậy cao.

Bậc 3 & 4: Áp dụng và Phân tích (Apply & Analyze)

- **Vai diễn của AI**: Người Hướng Dẫn Socratic (The Socratic Guide)
- **Chiến lược**: Chuyển từ “Biết gì” (What) sang “Dùng thế nào” (How) và “Tại sao” (Why).
- **Cách AI hoạt động**:
 - AI khởi tạo một bối cảnh (Scenario/Case Study) chứa các biến số thực tế và yêu cầu người học vận dụng kiến thức để giải quyết.
 - **Vòng lặp Socratic (Socratic Loop)**: Đây là thuật toán cốt lõi. Nếu người học trả lời sai, AI bị chặn (hard-coded) không được phép cung cấp đáp án. Thay vào đó, AI phải phân tích câu trả lời của người học, tìm ra lỗ hổng logic, và phản hồi bằng một *Câu hỏi gợi mở*.
 - *Ví dụ*: Nếu học viên áp dụng sai công thức, AI sẽ hỏi: “Nếu biến X tăng gấp đôi thì theo công thức của bạn, Y sẽ thế nào? Liệu điều đó có hợp lý trong thực tế không?” Vòng lặp này ép não bộ học viên phải tự tái cấu trúc mạng nơ-ron (Neuroplasticity).

Bậc 5 & Bậc 6: Đánh giá và Sáng tạo (Evaluate & Create)

- **Vai diễn của AI:** Luật Sư Của Quỷ (Devil's Advocate) & Đồng Ban Giám Khảo (Co-Evaluator)
 - **Chiến lược:** Đẩy người học ra khỏi vùng an toàn, ép họ phải bảo vệ quan điểm và tổng hợp tri thức để tạo ra cái mới.
 - **Cách AI hoạt động:**
 - **Tranh biện (Debate Mode - Bậc 5):** AI cố tình đưa ra một luận điểm sai lầm nhưng được nguy trang bằng các lập luận rất logic. Nhiệm vụ của người học là “bắt bẻ” AI. Nếu người học đuối lý, AI sẽ hạ cấp độ phức tạp của lập luận để mớm lời.
 - **Thiết kế (Design Mode - Bậc 6):** Người học phải nộp một dự án/kế hoạch mini. AI sẽ rà soát bản thiết kế này, chạy các bài test tưởng tượng (Edge-case testing) và phản hồi lại những lỗ hổng tiềm ẩn. Quá trình này mô phỏng môi trường làm việc thực tế, nơi kiến thức được nhào nặn thành sản phẩm.
-

3.3. Tích hợp Vòng lặp Micro-Feedback (Integration Loop)

Sự tương tác của AI Tutor không diễn ra trong không gian biệt lập mà được nối thẳng vào các thuật toán Toán học ở Phần 2. Mọi đoạn Text mà AI phản hồi đều đi kèm với một JSON payload ngầm:

4. **Gọi API Ebbinghaus:** Tính toán độ phức tạp của câu trả lời người học để cập nhật biến Performance (P), từ đó thay đổi Cường độ trí nhớ (S_{new}) ngay lập tức.
5. **Gọi API XP Engine:** Kích hoạt trigger cộng/trừ XP, hiển thị các hiệu ứng Gamification hạt (particle effects) như lửa (combo) hay rung lắc màn hình (khi sai) để neo giữ cảm xúc của học viên vào nền tảng.

Tóm lại, thông qua kiến trúc AI Context-Aware phân tầng theo Bloom, Knowledge Tree không chỉ dừng lại ở một công cụ quản lý nội dung học (LMS) mà đã tiến hóa thành một **Hệ sinh thái Gia sư Tư nhân Kỹ thuật số (Digital Private Tutoring Ecosystem)** có khả năng thấu hiểu và thích ứng vô hạn.